

# 내 일이 뭔지 아무도 모른다

에이전트에게 일을 넘기려면 그 일이 뭔지 적혀 있어야 한다. 그리고 적을 수 있는 사람은 그 일을 하는 당사자뿐이다.

|   |                                                  |          |
|---|--------------------------------------------------|----------|
|   | <b>한 장 요약</b>                                    | <b>3</b> |
| 1 | <b>무슨 문제인가</b><br>AI에게 넘길 일을 아무도 적어 놓지 않았다       | 4        |
| 2 | <b>그래서 무엇을 하려는가</b><br>AI가 할 부분과 사람이 할 부분을 갈라 준다 | 7        |
| 3 | <b>실제로 어떻게 생겼나</b><br>꺼내기·쪼개기·판정·실행              | 9        |
| 4 | <b>왜 이렇게 만들었나</b><br>결정 열 가지와 그 근거               | 13       |
| 5 | <b>실패한다면 어떻게 실패하나</b>                            | 17       |
| 6 | <b>왜 사서 쓰지 않고 만드나</b><br>Notion 템플릿으로 안 되는 것     | 19       |
| 7 | <b>언제 무엇이 나오나</b><br>2명, 먼저 한 부서                 | 21       |
| 8 | <b>지금 어디까지 왔나</b>                                | 23       |

|         |                                                                                                             |
|---------|-------------------------------------------------------------------------------------------------------------|
| 문제      | <p><b>AI에게 일을 넘기려는데, 무엇을 넘길지 아무도 적어 놓지 않았다.</b></p> <p>업무 지식은 당사자 기억 속에만 있고, 그걸 적어 둘 이유는 1년에 한두 번만 생긴다.</p> |
| 하는 일    | <p><b>업무를 단계로 꺼내서, AI가 대신할 수 있는 단계와 사람이 꼭 해야 하는 단계를 갈라 준다.</b></p> <p>기록이 목적이 아니다. 판정이 목적이고 기록은 그 재료다.</p>  |
| 대상      | <p><b>자기 업무를 줄글로도 못 쓰는 사람.</b></p> <p>쓸 수 있는 사람은 이미 Claude에 붙여넣으면 된다. 그건 잘된 일이고, 우리 고객이 아니다.</p>            |
| 차별점     | <p><b>판정 근거 셋 중 둘은 밖에서 알 수 없다.</b></p> <p>사내 도구에 연결이 되는지, 결재 규정상 사람이 필요한지. 일반 상식만 쓰면 Claude 래퍼가 된다.</p>     |
| 안 하는 것  | <p><b>사람을 줄 세우지 않는다.</b></p> <p>부서장은 팀원 문서를 볼 수 있다. 대신 개인별 합계·비율·비교를 아예 만들지 않는다.</p>                        |
| 가장 큰 위험 | <p><b>판정이 틀리는 것.</b></p> <p>"넘길 수 있다"고 해서 놀랐는데 안 되면 한 번에 끝이다. 그래서 정확도를 올리기 전에 틀렸을 때의 경로를 먼저 만든다.</p>        |
| 1차 목표   | <p><b>한 부서가 확실히 쓴다. 그 부서에서 실제로 넘어간 단계가 하나라도 나오는 것.</b></p> <p>전사 확산은 그다음 문제다.</p>                           |
| 지금      | <p><b>계산 부분만 완성. 화면은 아직 한 장도 없다.</b></p> <p>만드는 사람 2명. 사내에서 쓰고, 검증되면 오픈소스로 낸다.</p>                          |

# 무슨 문제인가

## 1장

회사가 AI로 업무를 자동화하기로 한다. 도구도 정했다. 그런데 바로 다음 질문에서 멈춘다. **무엇부터 자동화할 것인가.**

이 질문에 답할 수 있는 사람이 회사 안에 없다. IT팀은 시스템이 어떻게 도는지 알지만, 현업이 매주 세 시간씩 손으로 돌리는 엑셀 작업은 모른다. 반대로 그 일을 하는 사람은 "이게 자동화 대상이 될 수 있다"는 생각 자체를 해본 적이 없다. 그냥 원래 그렇게 하던 일이다.

## 에이전트 시대라서 이 문제가 더 커졌다

예전 자동화는 개발자가 붙었다. 그래서 무엇을 자동화할지도 개발자가 골랐고, 고를 수 있는 건 **시스템에 기록이 남는 일**뿐이었다.

에이전트는 다르다. 사람이 하던 일을 **사람이 설명한 대로** 할 수 있다. 범위가 갑자기 넓어졌다. 그런데 넓어진 그 영역이 정확히 아무데도 적혀 있지 않은 영역이다.

에이전트에게 시킬 수 있게 된 순간, 무엇을 시킬지 적어 놓는 일이 병목이 됐다.

## 왜 안 적혀 있나

회사 안에서 어떤 일이 어떻게 처리되는지 알아내는 방법은 지금까지 네 가지였다. 그리고 넷 다 같은 곳을 못 본다.

| 방법                     | 볼 수 있는 것             | 못 보는 것                                   |
|------------------------|----------------------|------------------------------------------|
| 시스템 기록 분석<br>Celonis 등 | 그룹웨어·전사시스템에 남은 처리 기록 | 기록이 안 남는 일. 도입 비용도 억 단위로 시작 자체가 어렵다      |
| 화면 녹화 분석<br>Scribe 등   | 브라우저 안에서의 클릭과 입력     | 브라우저 밖 전부. 전화, 대면 승인, 카카오톡, 옆 부서 회신 기다리기 |
| 그림 그리는 도구<br>Miro 등    | 사람이 직접 그린 그림         | 그림만 남고 데이터가 안 쌓인다. 에이전트가 읽을 수 있는 형태가 아니다 |
| 컨설팅                    | 인터뷰로 캐낸 진짜 흐름        | 한 번뿐이다. 프로젝트가 끝나는 날이 그 문서의 마지막 날이다       |

공통점을 보자. 어느 시스템에도, 어느 화면에도 남지 않는 일이 통째로 빠진다. 카카오톡으로 확인받고, 엑셀에 손으로 옮기고, 전화로 승인받고, 옆 부서 답을 기다리는 시간.

그런데 실무 시간의 상당 부분이 정확히 거기에 있다. 그리고 이 영역을 아는 사람은 **그 일을 하는 당사자 한 명**뿐이다. 기억 속에만 있다.

## 그럼 물어보면 되지 않나

맞다. 문제는 물어보는 방식이다. 지금까지 시도된 방식은 세 가지고, 셋 다 실패하는 이유가 분명하다.

|          |                                                                                      |
|----------|--------------------------------------------------------------------------------------|
| 설문지를 돌린다 | 표 칸을 채우게 하면 사람들은 정리된 요약을 쓴다. 정작 필요한 건 "이 단계에서 3일 대기가 생긴다" 같은 구체적인 사실인데, 그건 칸에 안 들어간다 |
|----------|--------------------------------------------------------------------------------------|

|            |                                                           |
|------------|-----------------------------------------------------------|
| 그림을 그리게 한다 | 빈 화면과 도형 팔레트를 주면 대부분 얼어붙는다. 업무를 아는 것과 도형으로 옮기는 것은 다른 능력이다 |
|------------|-----------------------------------------------------------|

|         |                                                    |
|---------|----------------------------------------------------|
| 인터뷰를 한다 | 제일 잘 나온다. 대신 한 사람당 한 시간이 든다. 그리고 인터뷰한 그날의 기록으로 굶는다 |
|---------|----------------------------------------------------|

## 그럼 Claude에 붙여넣으면 되지 않나

된다. 자기 업무를 줄글로 쓸 수 있는 사람은 그걸 붙여넣고 "어디를 자동화할 수 있어?"라고 물으면 오늘 당장 꽤 쓸 만한 답을 받는다. 공짜다.

이 사실을 피해 가면 안 된다. 이걸 못 넘으면 이 프로젝트는 만들 이유가 없다. 그래서 정직하게 따진다. 붙여넣기로 안 되는 게 뭔가.

| 붙여넣기로 안 되는 것  | 왜                                                                                            |
|---------------|----------------------------------------------------------------------------------------------|
| 애초에 붙여넣을 게 없다 | 대부분의 사람은 자기 업무를 줄글로도 못 쓴다. 몰라서가 아니라 순서와 조건으로 정리해 본 적이 없어서다. "붙여넣으면 된다"는 이미 쓸 수 있는 소수에게만 해당한다 |
| 우리 회사 도구를 모른다 | "그룹웨어에 등록한다"가 연결 가능한 단계인지는 우리 그룹웨어에 연결 통로가 있느냐에 달렸다. 그걸 아는 건 우리뿐이다                           |
| 우리 회사 규정을 모른다 | 어떤 단계는 기술적으로 가능해도 사람이 승인해야 한다. 결재 규정과 책임 범위는 회사마다 다르다                                        |
| 한 번 답하고 사라진다  | 대화는 안 남고 업무는 바뀐다. 그리고 답에서 실행까지 이어지지 않는다. "이렇게 하시면 됩니다"까지가 끝이다                                |

첫 줄이 가장 중요하다. 이 도구가 상대할 사람은 자기 업무를 줄글로도 못 쓰는 사람이다. 쓸 수 있는 사람은 이미 Claude를 쓰고 있고, 그건 잘된 일이다.

그래서 이 도구의 첫 번째 일은 분석이 아니라 꺼내는 것이다. 그 사람 머릿속에 있는 걸 질문으로 끄집어내서 단계로 만드는 것.

## 더 어려운 문제. 적을 이유가 없다

설령 도구가 아무리 쉬워도, 사람은 지금 안 아픈 일에 시간을 쓰지 않는다. 그리고 업무를 적어 두면 좋은 순간은 가끔, 갑자기 온다.

| 적어 두면 좋은 순간 | 얼마나 자주 오나 | 문제                         |
|-------------|-----------|----------------------------|
| 인수인계할 때     | 1~2년에 한 번 | 그때는 이미 늦다. 나가기 2주 전에 급히 쓴다 |
| 신입에게 설명할 때  | 연 1~2회    | 말로 하는 게 더 빠르다고 느낀다         |
| 휴가 대신할 사람에게 | 몇 달에 한 번  | 위와 같다                      |

#### 문제를 한 줄로

업무 지식은 당사자 기억 속에만 있는데, 그걸 적어 둘 이유는 1년에 한두 번만 생긴다. "쉽게 적게 해 주는 도구"만으로는 이 간극이 안 메워진다. 적는 그 순간에 바로 돌아오는 것이 있어야 한다.

## 단, 이 문제가 누구 문제인지는 정확히 하자

"내 일이 뭔지부터 알아야 한다"는 문제의식의 주인은 자동화를 추진하는 쪽이다. 영업팀 김대리가 아니다.

김대리는 자기 일을 안다. 몸으로 안다. 문서로 없을 뿐이다.

그에게 "당신 일이 뭔지 알아야 합니다"는 문제 제기가 아니라 잔소리다.

이 구분을 놓치면 제품 전체가 어긋난다. 그래서 다음 장은 "김대리가 지금 당장 뭘 얻는가"부터 시작한다.

# 그래서 무엇을 하려는가

## 2장

당신 업무를 단계로 꺼내서,  
AI가 대신할 수 있는 단계와 사람이 꼭 해야 하는 단계를 갈라 준다.

기록이 목적이 아니다. 판정이 목적이고 기록은 그 재료다. 이 순서를 뒤집으면 아무도 안 쓴다.

사람들이 문 앞에 서는 이유가 "업무를 기록해 두세요"가 아니라 "당신 일 중에 AI가 대신할 수 있는 걸 찾아 드릴게요"가 된다. 전자는 나중을 위한 저축이고, 후자는 지금 궁금한 것이다.

### 판정은 세 갈래다

#### 넘길 수 있다

연결 통로가 있고 규정에도 안 걸린다.

"메일에서 요청 내용을 뽑아 표에 옮긴다"

#### 조건부

기술은 되는데 사람 확인이 필요하다.

"초안까지 만들고 보내기 전에 확인받는다"

#### 사람이 해야

규정·책임·판단이 걸려 있다.

"금액 승인", "고객에게 사과 전화"

가운데 칸이 제일 중요하다. 실무는 대부분 여기서다. 통째로 넘기는 것도, 통째로 사람이 하는 것도 아니고 일부만 넘기고 사람이 확인하는 형태가 현실적이다. 이 칸이 없으면 판정이 극단으로 갈리고, 극단으로 갈린 판정은 안 맞는다.

### 무엇을 근거로 판정하나

여기가 이 도구의 존재 이유다. 셋을 쓰는데, 둘은 Claude가 원리상 알 수 없다.

| 판정 근거          | 밖에서 아나 | 내용                                                                      |
|----------------|--------|-------------------------------------------------------------------------|
| 사내 도구 연결 가능 여부 | 모름     | 단계에 나온 도구가 우리가 붙일 수 있는 것인지. 우리 회사 도구 목록과 연결 가능 여부를 미리 조사해서 표로 들고 있어야 한다 |
| 규정상 사람이 필요한가   | 모름     | 결재 권한, 법적 책임, 대외 커뮤니케이션. 이건 회사가 한 번 정해서 넣어 두는 규칙이다                      |
| 일반적인 판단        | 얕      | 이 종류의 작업이 대체로 자동화 가능한지. 이것만 쓰면 Claude 래퍼가 된다. 앞의 둘과 겹쳐야 값어치가 생긴다        |

#### 나중에 가장 강해지는 근거

네 번째가 있다. "이미 다른 부서가 이 단계를 넘겼나." 초기에는 쌓인 게 없어 못 쓰지만, 쓰일수록 강해지고 원리상 밖에서는 절대 알 수 없다. "총무팀에서 이미 이 단계 넘겼어요. 그거 그대로 쓰실래요?"

## 판정이 끝이 아니라 시작이다

"이렇게 하시면 됩니다"까지가 끝나면 붙여넣기와 다를 게 없다. 그래서 판정 옆에 **실행 버튼**이 붙는다.

"넘길 수 있다"로 판정된 단계에 "지금 해볼까요?"가 뜬다. 누르면 그 단계만 에이전트가 대신 해 본다. 전체 자동화가 아니라 한 단계다. 되면 다음부터 그 단계는 안 해도 되고, 안 되면 판정이 틀렸다는 게 즉시 드러난다.

## 그러면 기록은 왜 남기나

판정을 받으려면 어차피 업무를 단계로 꺼내야 한다. 그 과정에서 나온 것이 **부산물로 남는다**. 그리고 그 부산물이 쓸모가 있다.

- 다음에 그 일 할 때 체크리스트로 연다. 빠뜨리지 않고, 어디까지 했는지 안다. 그리고 열 때마다 틀린 단계가 그 자리에서 고쳐진다
- 인수인계·신입 교육에 그대로 쓴다. 급하게 워드를 켜지 않아도 된다
- 회사는 자동화 후보 지도를 얻는다. 짐작이 아니라 당사자 기록에서 나온다

셋 다 값어치가 있지만, 셋 중 어느 것도 처음 문을 열게 만들지는 못한다. 1장에서 본 대로 이 이득들은 가끔 오기 때문이다. 문을 여는 건 판정이고, 나머지는 그 뒤에 따라온다.

## 누가 무엇을 볼 수 있나

사내 도구이므로 부서장은 팀원 문서를 볼 수 있다. 숨기지 않는다. 첫 화면에 그렇게 적힌다. 대신 네 가지를 함께 건다.

|       |                                            |
|-------|--------------------------------------------|
| 서로 본다 | 부서장이 팀원 걸 보면 팀원도 부서장 걸 본다. 그리고 부서장이 먼저 적는다 |
|-------|--------------------------------------------|

|         |                                           |
|---------|-------------------------------------------|
| 열람이 남는다 | "부서장님이 어제 이 문서를 보셨습니다." 보는 건 자유지만 몰래는 아니다 |
|---------|-------------------------------------------|

|           |                                                            |
|-----------|------------------------------------------------------------|
| 줄 세우지 않는다 | 개인별 소요시간 합계·근무시간 대비 비율·사람 간 비교를 아예 만들지 않는다. 문서는 보이되 점수는 없다 |
|-----------|------------------------------------------------------------|

|        |                                        |
|--------|----------------------------------------|
| 메모는 따로 | 공유용 문서와 비공개 메모는 저장되는 곳이 다르다. 섞일 경로가 없다 |
|--------|----------------------------------------|

정직하게 적어 둔다. 이 구조는 1장의 "사실대로 안 적는다" 문제를 **완전히 해결하지 못한다**. 완화할 뿐이다. 그래서 소요시간은 부풀려질 수 있다고 전제하고 설계한다(4장 여섯 번째 결정). 자발 참여이고 처음에는 한 부서라서, 그 위험을 감당할 수 있다고 본다.



# 실제로 어떻게 생겼나

3장 · 총무팀의 구매 요청 처리를 예로 든다

다섯 단계다. 꺼내기 → 쪼개기 → 판정 → 실행 → 다음에 다시 열기. 사용자는 선을 단 한 번도 직접 긋지 않는다.

## ① 못 쓰는 사람에게서 꺼낸다

1장에서 정한 대로, 이 도구가 상대할 사람은 자기 업무를 줄글로 못 쓰는 사람이다. 그래서 빈 화면을 주지 않는다. 한 번에 한 질문만 한다.

2 / 5

그다음엔 뭘 하세요?

확인한다    누구한테 물어본다    기다린다    직접 입력한다

고르거나, 그냥 적으셔도 됩니다

계층도 순서도 없다. 틀릴 수 있는 구석이 없어야 한다

붙여넣을 게 있으면 그게 더 빠르다. 기존 워드 매뉴얼이나 카카오톡 인수인계를 그대로 붙여넣으면 한국어 어미와 번호 패턴으로 단계를 쪼갬다. 비슷한 업무를 이미 누가 적어 뒀으면 복사해서 고치는 게 가장 빠르다.

## ② 단계로 쪼개지고 그림이 그려진다

한 줄에 한 단계씩 정리된다. 갈라지는 곳에서는 **주황색 부분**을 화면이 대신 인쇄해 준다. 사용자는 빈칸만 채운다.

● ● ● 구매 요청 처리 · 총무팀

1 구매 요청서가 메일로 온다  
2 요청 내용이 규정에 맞는지 확인한다  
3 만약 100만원 미만이면 바로 승인한다  
4 만약 100만원 이상이면 팀장님 결재를 기다린다  
5 구매 시스템에 등록한다  
6 발주하고 요청자에게 알린다

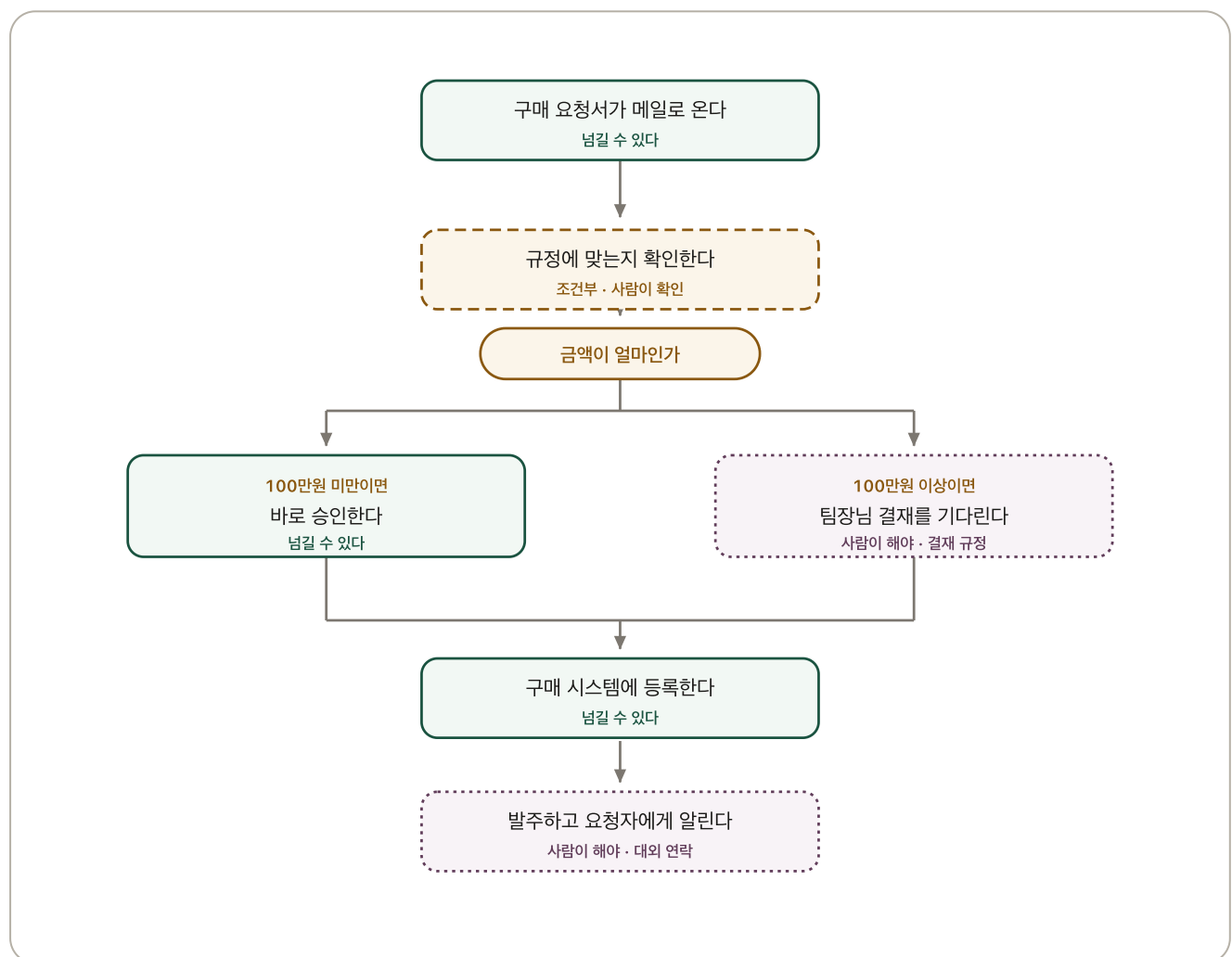
도형도, 선도, 메뉴도 없다

단계는 세 가지 모양 중 하나가 된다. 타이틀을 늘리면 배울 게 늘고, 안 늘리면 실제 업무를 못 그린다.

|     |                      |                                                                      |
|-----|----------------------|----------------------------------------------------------------------|
| 모양  | 무엇인가                 | 왜 나눠야 하나                                                             |
| 할 일 | 내가 직접 하는 일           | 대부분이 여기다                                                             |
| 갈래  | 상황에 따라 길이 나뉘는 곳      | "둘 중 하나"와 "둘 다 동시에"를 구분해야 한다. 안 하면 숫자가 틀린다                           |
| 기다림 | 내가 아무것도 못 하고 기다리는 시간 | 대부분의 업무 도구는 사람이 손을 움직이는 시간만 센다. 위 예시에서 손이 가는 시간은 20분인데 끝나기까지 3일이 걸린다 |

### ③ 여기서부터가 이 도구의 값어치다

각 단계에 판정이 없인다. 색만으로 구분하지 않고 테두리 모양도 다르게 해서, 색을 구분하기 어려운 사람도 알 수 있게 한다.



같은 흐름도인데 이제 무엇을 넘길 수 있는지가 보인다

#### 모양과 판정은 다른 축이다

헛갈리기 쉬운데, **단계의 모양**(할 일·갈래·기다림)과 **판정**(넘길 수 있다·조건부·사람이 해야)은 서로 다른 질문에 대한 답이다. 모양은 "이게 무슨 종류의 일인가", 판정은 "이걸 SI에게 넘길 수 있나"다.

|     | 넘길 수 있다     | 조건부           | 사람이 해야       |
|-----|-------------|---------------|--------------|
| 할 일 | 값을 옮겨 담는 일  | 초안 쓰기         | 고객에게 사과 전화   |
| 갈래  | 금액으로 갈리는 조건 | 애매한 판단이 끼는 조건 | 사람이 봐야 아는 조건 |
| 기다림 | 자동 재촉 알림    | 기한 넘으면 알림     | 팀장 결재 대기     |

위 흐름도의 4단계가 좋은 예다. 모양은 "기다림"이고 판정은 "사람이 해야"다. 기다림이라고 다 못 넘기는 게 아니고, 할 일이라고 다 넘길 수 있는 것도 아니다.

#### ④ 왜 그렇게 판정했는지 펼쳐 보여준다

근거를 안 보여주면 아무도 안 믿는다. 그리고 틀렸을 때 고칠 방법도 없다. 그래서 단계를 누르면 판정 근거가 펼쳐진다.

##### 5단계 · 구매 시스템에 등록한다

판정: 넘길 수 있다

**확인** 이 단계에 나온 구매 시스템은 우리 회사에서 연결 가능한 도구다

**확인** 결재 규정에 사람이 직접 해야 한다는 조항이 없다

**확인** 정해진 값을 옮겨 담는 작업이라 판단이 끼지 않는다

**아직** 이 단계를 넘긴 다른 부서는 아직 없다

##### 이 판정이 틀렸다면

"아니에요, 이건 사람이 해야 해요"를 누르면 판정이 바뀌고 **이유**를 한 줄 적게 한다. 같은 이유가 여러 번 쌓이면 그건 규칙이 잘못됐다는 뜻이라 규칙을 고친다.

판정보다 근거가 중요하다. 근거가 없으면 두 번째부터 아무도 안 본다

#### ⑤ 한 단계씩 실제로 넘겨 본다

"이렇게 하시면 됩니다"까지가 끝이면 붙여넣기와 다를 게 없다. "넘길 수 있다"로 판정된 단계에 "지금 해볼까요?"가 뜬다.

전체 자동화가 아니라 한 단계다. 되면 다음부터 그 단계는 안 해도 되고, 안 되면 판정이 틀렸다는 게 그 자리에서 드러난다. 이 실패가 곧 다음 판정을 정확하게 만드는 재료가 된다.

## ㉔ 다음에 그 일 할 때 다시 연다

남은 단계는 여전히 사람이 한다. 그래서 다음에 그 일을 할 때 이 문서를 체크리스트로 연다. 빠뜨리지 않고, 중간에 끊겨도 어디까지 했는지 안다.

### 이게 왜 중요한가

문서가 스스로 정확해진다. 열었는데 단계가 틀리면 그 자리에서 고친다. 아무도 "문서 갱신하세요"라고 시키지 않아도 된다.

그리고 다시 열 때마다 판정도 다시 돈다. 지난달엔 못 넘기던 단계가 새 연결이 생겨 넘길 수 있게 되면, 그때 "이제 이거 넘길 수 있어요"가 뜬다. 돌아올 이유가 계속 생긴다.

# 왜 이렇게 만들었나

## 4장 · 결정 열 가지

중요한 순서다. 첫 번째가 나머지 전부를 지배한다. 각 결정에는 어떤 신호가 오면 다시 논의할지가 함께 적혀 있다. 그걸 미리 적어 두지 않으면, 우리가 무엇을 가정하고 있는지 스스로도 모르게 된다.

판정 이 도구가 죽고 사는 곳

1 · 2

### 하나 · 판정이 틀린다는 전제로 설계한다

이 도구는 판정 때문에 쓰인다. 그러니 판정이 틀리면 끝이다. "이건 AI가 할 수 있어요" 했는데 실제로 못 하면, 그 사람은 두 번 다시 안 온다. 한 번이면 충분하다.

그래서 정확도를 올리려 애쓰는 대신 틀렸을 때의 경로를 먼저 만든다.

- 근거를 항상 펼쳐 보여준다. 판정만 던지면 믿을 수도 반박할 수도 없다
- 사용자가 뒤집을 수 있고, 뒤집으면 이유를 한 줄 받는다. 같은 이유가 여러 번 쌓이면 규칙이 잘못된 것이므로 규칙을 고친다
- "조건부" 칸을 넉넉히 쓴다. 애매하면 조건부다. "넘길 수 있다"고 잘못 말하는 것이 "조건부"라고 말하는 것보다 훨씬 싸다
- 한 단계씩만 실제로 넘겨 본다. 판정이 맞았는지 그 자리에서 판명된다

**비대칭이 핵심이다.** 못 넘길 걸 못 넘긴다고 하면 아무 일도 안 일어난다. 넘길 수 있다고 했는데 안 되면 신뢰가 통째로 날아간다. 그래서 판정은 의도적으로 보수적으로 기울인다. 성능 지표가 아니라 신뢰 지표를 본다.

### 둘 · 부서장은 다 본다. 대신 네 가지를 건다

사내 도구이므로 부서장은 팀원 문서를 볼 수 있다. 숨기지 않고 첫 화면에 그렇게 적는다. "안 봅니다"라고 해 놓고 보는 것이 제일 나쁘고, 그건 반드시 들킨다.

대신 2장에 적은 넷을 함께 건다. 서로 본다 · 열람이 남는다 · 줄 세우지 않는다 · 메모는 따로. 특히 세 번째가 실질적인 방어선이다.

**"문서를 본다"와 "점수를 본다"는 완전히 다른 문제다.** 개인별 소요시간 합계, 근무시간 대비 비율, 사람 간 비교를 아예 만들지 않는다. 원본은 보이되 줄을 세울 재료를 안 준다. 이게 없으면 도구가 인사평가 자료로 흘러가고, 그때부터 아무도 사실대로 안 적는다.

**대가를 알고 택한다.** 이 구조는 1장의 "사실대로 안 적는다" 문제를 완전히 해결하지 못한다. 자발 참여이고 처음엔 한 부서라서 감당 가능하다고 보는 것이지, 풀었다고 보는 게 아니다. 뒤집는 조건은 이렇다. 확산 단계에서 부서장 열람이 참여를 막는 신호가 관측되면 그때 닫는다. 다만 닫는 건 여는 것보다 훨씬 어렵다는 걸 알고 있다.

### 셋 · 들어쓰기로 갈래를 만들게 하지 않는다

처음 설계는 Tab 키로 단계를 안쪽으로 밀어 넣어 갈래를 표현하는 방식이었다. 개발자에게는 자연스럽지만, **전체 직원의 30~50%는 이 개념을 바로 이해하지 못한다고 봤다.** 그래서 대신 "만약 ~라면"이라는 말을 화면이 직접 인쇄해 주고, 사용자는 빈칸만 채운다.

더 나쁜 건 **실패해도 조용하다**는 점이다. Tab을 안 눌러도 화면은 멀쩡히 그려진다. 사용자는 자기가 잘못 적었다는 걸 모른 채 나중에 "이 도구 이상하다"고 결론 내린다. 요란하게 실패하는 건 고칠 수 있지만, 조용히 실패하는 건 신뢰만 깎아먹는다.

### 넷 · 단계의 모양은 셋으로 고정한다

할 일 · 갈래 · 기다림(3장). 여기서 늘리지 않고, 나머지 차이는 각 단계에 붙는 정보로 흡수한다.

특히 갈래에서 "둘 중 하나"와 "둘 다 동시에"를 구분하는 게 중요하다. 구분하지 않으면 그림만 어긋나는 게 아니라 숫자가 틀린다.

실제로 재 봤다. 같은 그림에서 두 갈래가 **동시에** 진행되면 총 4시간, **둘 중 하나만** 가면 평균 2.5시간인데, 구분 없이 그냥 더하면 5시간이 나온다. 각각 25%, 100% 부풀려진 값이다. 이 숫자로 우선순위를 정하면 순위가 뒤집힌다.

### 다섯 · 쓰는 도중에는 아무것도 묻지 않는다

쓰는 흐름 안에 시간·도구·담당자 같은 항목을 노출하면, 담당자 본인 외에는 **20%도 채우지 않는다.** 그래서 다 쓴 뒤로 미루고, 입력이 아니라 확인으로 바꾼다. "엑셀에 정리"라고 쓰면 도구를 알아서 짐작해 주고, 사용자는 맞는지만 눌러 준다.

안 채우는 건 게을러서가 아니다. 셋이다. 기억을 되짚는 중에 숫자 질문이 끼어들어 흐름이 끊기고("인지 전환 비용"), 정확히 몰라서 못 적고("30분인지 40분인지 모르겠는데"), 채워도 나에게 돌아오는 게 없다.

## 측정 숫자를 믿을 수 있게

### 여섯 · 기억이 틀린다는 걸 전제로 묻는다

사람의 기억은 시간보다 횟수에서 더 크게 틀린다. 그런데 업무량은 시간 × 횟수라서 두 오차가 곱해진다. 그래서 순위 맨 위가 통째로 뒤집힌다. 대응은 앞서 본 대로다. "평소 얼마나 자주" 대신 "**지난 7일 동안 몇 번**".

가장 효과가 큰 장치는 **총량을 되돌려 보여주는 것이다.** 그 사람이 적은 모든 업무의 월 합계를 계산해서 보여준다. "**합계 240시간 / 한 달 근무 160시간**". 그러면 본인이 알아서 다시 나눈다. 개별 숫자를 백 번 다듬는 것보다 이 한 화면이 낫다.

### 일곱 · 자동화보다 제거를 먼저 묻는다

순위를 매기기 전에 먼저 묻는다. "자동화하기 전에, 이거 그냥 없애면 안 되나?" 승인 세 단계 중 반려율이 0%인 단계는 자동화 대상이 아니라 없앨 대상이다.

제거 후보는 자동화 후보보다 **훨씬 드물게 발견되고, 경영진에게 훨씬 잘 팔린다**. 그리고 **개발이 0줄이다**. 3장 마지막 카드가 실제 예다. 결재 기준선을 올리는 것만으로 3일이 줄어든다.

### 여덟 · 남에 대한 평가는 수집하지 않는다

묻는 건 "나는 여기서 기다린다"뿐이다. "저쪽이 얼마나 늦나요"는 묻지 않는다.

자발 참여라서 더 그렇다. 원하는 사람만 쓰는 도구에 상대를 평가하는 칸이 하나라도 있으면, **그 칸이 있다는 사실만으로 안 쓸 이유가 생긴다**.

대신 한 사람의 기록만으로도 기다림이 쌓이는 지점이 그대로 드러난다. 상대 부서가 한 글자도 안 적어도 나온다.

초안은 "여러 부서 기록을 이어 붙여 어긋난 지점을 찾는다"를 핵심 가치로 뒀는데, 자발 참여에서는 성립하지 않는다. 한 업무가 네 부서에 걸쳐 있고 참여율이 40%면 **양쪽이 다 걸릴 확률이 16%밖에 안 된다**. 6건 중 1건 나오는 걸 핵심 가치라 부를 수 없다. 그래서 **한쪽만으로 작동하는 쪽으로 낮춰 잡았다**. 양쪽이 다 적히면 나오는 건 덤이다.

함정이 하나 있다. "구매 승인을 기다린다"는 기록이 여러 명에게서 모이면 결과가 사실상 **"구매팀이 느리다"**로 읽힌다. 자기 사실만 모았는데 남의 평가가 되는 것이다. 그래서 집계 단위를 부서가 아니라 **이음매**로 잡는다. "구매팀"이 아니라 "구매 승인이라는 이음매에서 평균 3일이 쌓인다"로.

### 아홉 · 디자인은 마감이 아니라 기능이다

이유가 둘이다. "회사가 시킨 시스템"처럼 보이면 그 순간 끝난다. 감시 도구라는 인상은 문구보다 보이는 느낌에서 먼저 결정된다. 그리고 **결과물이 남에게 공유된다**. 보기 좋아야 공유하고, 공유돼야 퍼지므로 디자인이 곧 확산 속도다.

그래서 이런 것들을 구체적으로 금지했다. **달성률 게이지·순위표·팀 평균**(집계는 아예 그리지 않는다), **빨강/노랑/초록 신호등**(색이 판단하면 안 된다. 분류만 한다), **완료 축하 효과**(자기 일을 없앴다는 불안을 정확히 자극한다), **파란색**("회사 시스템" 인상을 준다).

## 열 · 오픈소스로 내도 프라이버시 규칙은 설정으로 빼지 않는다

부서장 위로는 업무 단위 집계만 보이고, 그 집계도 5명보다 적은 그룹은 표시되지 않는다. 3명짜리 그룹의 평균은 사실상 개인 정보이기 때문이다.

이 도구는 나중에 공개할 계획이다(6장). 그러면 회사마다 사정이 다르니 그 기준을 설정으로 빼 달라는 요구가 반드시 온다. 규모가 작은 곳에서는 5명 기준이 거의 모든 집계를 막아 버리니 합리적인 요구다.

그래도 빼지 않는다. 코드에 숫자로 박아 두고, 낮추려면 갈라내서 고치게 한다. 그러면 누가 언제 낮췄는지 기록에 남는다.

---

두 번째 결정에서 이미 **부서장 열람을 허용했다**. 남은 방어선이 이것뿐인데 그것마저 설정으로 빼면 아무것도 안 남는다.

"오픈소스에는 설정이 있던데요" 한마디에 무너진다. **공개판에 존재하는 설정은 사내에도 존재하는 설정이다**.

---

대가는 분명하다. **작은 회사는 이 도구를 그대로 못 쓴다**. 감수한다. 사내에서 쓰려고 만드는 도구이고 공개는 덤이다. 덤을 위해 본체를 무르지 않는다.



# 실패한다면 어떻게 실패하나

## 5장

기능이 부족해서가 아니다. 위험한 순서대로 적는다.

| 이렇게 죽는다            | 무슨 일이 벌어지나                                               | 막는 방법                                                                   |
|--------------------|----------------------------------------------------------|-------------------------------------------------------------------------|
| 하나<br>판정이 틀린다      | "넘길 수 있다"고 해서 눌렀는데 안 된다. 한 번이면 끝이다. 그 사람은 두 번 다시 안 온다    | 4장 첫 번째 결정 전체. 근거를 항상 펼치고, 애매하면 "조건부"로 기술키고, 틀렸을 때 뒤집는 경로를 먼저 만든다       |
| 둘<br>Claude 래퍼가 된다 | 판정 근거가 일반 상식뿐이면 붙여넣기와 결과가 같다. 그러면 굳이 이걸 쓸 이유가 없다         | 사내 도구 연결 표와 결재 규정을 먼저 채워 둔다. 이 둘이 비어 있으면 열지 않는다                         |
| 셋<br>첫 문장을 못 쓴다    | 대상이 줄글로 못 쓰는 사람이다. 빈 화면을 주면 그대로 달는다                      | 한 번에 한 질문. 붙여넣기 지원. 비슷한 업무 복제. 백지를 아예 안 준다                              |
| 넷<br>감시 도구로 찍힌다    | 부서장이 다 보므로 이 위험이 구조적으로 남아 있다. 한 번 찍히면 정책을 바꿔도 신뢰는 안 돌아온다 | 부서장이 먼저 적는다. 열람 기록을 작성자에게 보여준다. 줄 세울 재료를 안 만든다. 안 적는 부서장이 있는 부서에는 안 넣는다 |
| 다섯<br>넘긴 게 없다      | 판정만 나오고 실제로 넘긴 단계가 0건이면, 결국 아무것도 안 바뀐 것이다                | 가장 쉬운 한 단계부터 실제로 붙인다. 1단계 안에 실제로 넘어간 단계가 하나는 있어야 한다                     |
| 여섯<br>그림이 춤춘다      | 타이핑할 때마다 도형이 움직이면 즉시 신뢰를 잃는다                             | 글자를 고치는 중에는 배치를 다시 계산하지 않는다. 단계가 추가·삭제될 때만 다시 그린다                       |

### 숫자 하나만 본다면 이것

실제로 넘어간 단계의 수. 문서를 몇 개 만들었는지, 몇 명이 썼는지는 이것 앞에서 아무 의미가 없다. 한 단계라도 실제로 AI가 대신하기 시작하면 그때부터 이 도구는 스스로 팔린다. 그 전까지는 아무리 잘 만들어도 "설명은 좋은데 안 쓰는 도구"다.

## 무엇을 재는가

1차 목표가 한 부서이므로 전사 지표는 아직 안 본다. 그 부서 안에서 이 여섯 개를 본다.

| 재는 것             | 목표     | 못 맞추면 무슨 뜻인가      |
|------------------|--------|-------------------|
| 실제로 넘어간 단계       | 1건 이상  | 판정이 말뿐이다          |
| 판정을 뒤집는 비율       | 20% 이하 | 판정이 안 맞는다         |
| 그 부서에서 문서를 만든 사람 | 절반 이상  | 첫 문장을 못 쓰고 있다     |
| 첫 판정까지 걸리는 시간    | 10분 이내 | 꺼내는 과정이 너무 무겁다    |
| 그 일 할 때 다시 여는 비율 | 25%    | 죽은 문서만 쌓인다        |
| 부서장이 자기 걸 적었나    | 필수     | 상호성이 깨져 감시 도구가 된다 |

두 번째가 가장 정직한 지표다. 판정을 뒤집는 비율은 사용자가 우리 판단을 얼마나 안 믿는지를 직접 보여준다. 그리고 뒤집을 때 받는 한 줄이 다음 판정을 고치는 재료가 된다.

# 왜 사서 쓰지 않고 만드나

## 6장

이건 밖에 파는 제품이 아니다. 우리 회사에서 쓰려고 만든다. 그러니 답할 질문도 다르다. "경쟁에서 이기나"가 아니라 "직접 만들 값어치가 있나"다.

## 가장 그럴듯한 대안은 Notion이다

직원이 이미 글을 쓰고 있는 곳이고, 인공지능이 흐름도를 그려 주며, 표 속성으로 담당자·도구·시간을 담고 합계까지 낸다. 템플릿 한 장이면 이 제품이 하려는 일의 70%가 재현된다. 2명이 6개월을 쓰는 것보다 훨씬 싸다.

그래서 정직하게 따져 봤다. 템플릿으로 안 되는 게 뭔가.

| 필요한 것          | 템플릿으로 되나                                                                 |
|----------------|--------------------------------------------------------------------------|
| 회사 맥락으로 판정한다   | 안 된다. 우리 도구에 연결이 되는지, 규정상 사람이 필요한지를 Notion도 Claude도 모른다. 이게 이 도구의 존재 이유다 |
| 판정에서 실행까지 이어진다 | 안 된다. "이렇게 하시면 됩니다"까지가 끝이다. 단계 하나를 눌러서 실제로 넘기는 경로가 없다                    |
| 글과 그림이 같이 움직인다 | 안 된다. 한 번 그려 주고 끝이라 3주 뒤 업무가 바뀌면 글만 고쳐지고 그림은 그대로 남는다. 그러면 아무도 그림을 안 믿는다  |
| 줄 세우지 않는다      | 안 된다. 관리자 권한으로 다 보이고, 보이는 걸 가공해 사람을 비교하는 걸 막을 방법이 없다                     |
| 쓰기 쉽다          | 템플릿이 더 낫다. 이미 쓰는 도구라 새로 배울 게 없다. 이건 우리가 진 부분이고, 3장 전체가 이 격차를 메우려는 시도다    |

결론은 이렇다. 처음 두 줄 때문에 만든다. 회사 맥락으로 판정하는 것, 그리고 판정에서 실행까지 이어지는 것. 나머지는 템플릿으로도 어느 정도 된다.

그래서 1단계에서 그 두 줄이 안 되면 이 프로젝트는 접는 게 맞다. 나머지를 아무리 잘 만들어도 Notion 템플릿이나 Claude 대화보다 나을 이유가 없기 때문이다.

## 이미 있는 도구로는 왜 안 되나

화면 녹화 방식(Scribe 등)은 입력 비용에서 우리보다 낫다. 적지 않아도 기록되니까. 다만 1장에서 본 대로 브라우저 밖을 못 본다. 전화, 대면 승인, 카카오톡, 옆 부서 회신 대기가 통째로 빠진다.

시스템 기록 분석(Celonis 등)은 도입 비용이 억 단위다. 300~1,000명 규모에서 검증되지 않은 문제에 쓸 금액이 아니다.

## 사내에서 뭐라고 소개할 것인가

같은 도구를 두 가지로 소개한다. 듣는 사람이 다르기 때문이다. 이건 마케팅이 아니라 정직함의 문제다. 직원에게 "자동화하려고요"라고 말하면 1장에서 본 데이터 오염이 그대로 일어난다.

경영진에게 "직원들이 직접 만드는, 우리 회사의 자동화 후보 목록"

직원에게 "쓰기만 하면 흐름도가 되는 업무 기록. 인수인계와 신입 교육용"

직원용 소개만으로는 만들 명분이 없고, 경영진용 소개만으로는 쓰는 사람이 없다.

둘 다 사실이라서 가능한 소개다. 어느 쪽에도 없는 말을 지어내지 않는다. 그리고 직원에게 한 약속이 구조로 지켜지므로(4장 첫 번째 결정), 두 소개가 나중에 충돌하지 않는다.

## 오픈소스로 낼 계획이다

사내에서 쓸 만하다고 검증된 뒤 공개한다. 검증되지 않은 걸 내놓아도 아무도 안 쓰고, 우리도 배울 게 없다.

다만 그 결정이 지금부터 설계를 제약한다. 나중에 발라내려면 훨씬 비싸기 때문이다.

- 사내 데이터는 코드에 안 들어간다. 부서명·도구명·인터뷰로 채운 예시 업무는 전부 별도 데이터로 분리한다. 7장 0단계에서 우리가 대신 적는 8~12개 부서 기록이 바로 여기 걸린다
- 프라이버시 규칙은 설정으로 빼지 않는다. 이유는 4장 열 번째 결정에 있다
- 계산 부분은 이미 독립적이다. 화면이나 서버에 기대지 않고 함수 호출만으로 동작한다. 지금 완성된 부분이 그대로 공개 대상이 된다

# 언제 무엇이 나오나

7장 · 2명, 먼저 한 부서

## 먼저 증명할 것은 딱 하나다

자기 업무를 줄글로 못 쓰는 사람이 도움 없이 10분 안에 업무 하나를 꺼내고, 나온 판정을 보고 "이건 넘겨도 되겠다" 싶은 단계를 실제로 한 번 넘긴다.

"넘겼다"까지가 가설이다. 판정이 나왔다는 것만으로는 아무것도 증명되지 않는다

| 단계  | 이때 사용자가 할 수 있게 되는 것                                                              |
|-----|----------------------------------------------------------------------------------|
| 1단계 | 질문에 답해서 업무를 꺼내고, 흐름도를 보고, 판정을 받고, 가장 쉬운 한 단계를 실제로 넘긴다. 여기까지가 가설 전체다              |
| 2단계 | 다음에 그 일 할 때 체크리스트로 연다. 열 때마다 판정이 다시 돌아서, 새로 넘길 수 있게 된 단계를 알려 준다. 돌아올 이유가 여기서 생긴다 |
| 3단계 | 넘길 수 있는 단계가 늘어난다. 사내 도구 연결을 하나씩 붙이고, 결재 규정을 규칙으로 채운다                             |
| 4단계 | 인수인계 문서·신입 교육 자료 내보내기. 공유 링크                                                     |
| 5단계 | 부서 단위 요약. 순위표가 아니라 요약이다. 여기서 제거 후보가 나온다                                          |

초안은 이 순서를 만들기 쉬운 순으로 짰는데 잘못이었다. 판정과 실행이 뒤에 있으면 아무도 1단계를 안 쓴다. 그래서 쓰게 되는 순으로 다시 짰고, 그 결과 1단계가 가장 무거워졌다. 그게 맞다. 가설이 거기 있기 때문이다.

### 1단계를 열기 전에 반드시 있어야 하는 것

사내 도구 연결 표와 결재 규정 규칙. 이 둘이 비어 있으면 판정 근거가 일반 상식뿐이고, 그러면 5장 두 번째 위험 (Claude 래퍼)이 그대로 실현된다. 도구를 만드는 것보다 이 표를 채우는 게 먼저다.

## 일정에 대해 정직하게

2명 기준 약 6개월을 잡는다. 1단계만 해도 판정 규칙과 실행 연결이 들어가서 초안보다 무겁다. 그리고 우리 문서 안에 반증이 이미 있었다. 화면 부품 16개를 만드는 데만 5주를 잡아 둔 문서가 따로 있는데, 초안은 전체를 3.5주로 잡고 있었다.

## 에이전트를 언제 본격적으로 붙이나

시점을 미리 정하지 않는다. 대신 착수 신호를 정해 둔다. 1단계에서 한 단계씩 넘겨 보는 것까지는 하되, 본격적인 연결은 이 신호가 켜지면 시작한다.

신호 하나

"넘길 수 있다" 판정을 받은 단계가 10건 넘게 쌓였는데 실제로 넘어간 건 2건 이하일 때. 판정은 되는데 실행이 안 붙고 있다는 뜻이다

신호 둘

사용자가 "이거 그냥 자동으로 해 주면 안 되나요"를 먼저 말할 때. 요구가 우리보다 앞서 있다는 뜻이다

신호 셋

같은 단계를 여러 사람이 반복해서 넘기려 할 때. 한 번 붙이면 여러 명이 쓴다

왜 시점 대신 신호인가. 지금은 얼마나 걸릴지 모르고, 모르는 걸 날짜로 적으면 그 날짜가 지날 때 계획 전체가 안 믿기게 된다. 신호는 지나가도 계획이 안 무너진다. 아직 안 커졌을 뿐이다.

## 어느 부서부터 여나

업무 성격이 아니라 사람으로 고른다. 가장 협조적인 부서부터 간다. 부서장이 자기 업무를 먼저 적어 줄 부서여야 한다. 이건 조건이지 선호가 아니다.

대신 이 선택의 대가를 알고 있어야 한다. 협조적인 곳에서 성공한 건 일반화가 안 된다. 그래서 두 번째 부서는 일부러 회의적인 곳을 고른다. 거기서도 되면 진짜다.

그리고 열기 전에 우리가 먼저 채운다. 8~12개 업무를 우리가 직접 인터뷰해서 대신 적어 둔다. 빈 서비스 문제는 사용자가 아니라 우리가 푼다. 그래야 "비슷한 업무 복제"가 첫날부터 작동한다.

# 지금 어디까지 왔나

## 8장

솔직하게 먼저 말한다. 지금 브라우저를 열어서 쓸 수 있는 건 아무것도 없다. 3장에서 본 화면들은 이 문서를 위해 그린 것이고, 실제 제품으로는 아직 존재하지 않는다.

그리고 이번에 방향을 바꿨으므로, 이미 만들어 둔 것 중 무엇이 살아남는지부터 적는다. 이걸 안 적으면 "다 만들어 났다"는 인상만 남는다.

| 만들어 둔 것       | 새 방향에서 | 왜                                                                                  |
|---------------|--------|------------------------------------------------------------------------------------|
| 글을 단계로 쪼개기    | 그대로 쓴다 | 꺼내기의 핵심이다. 오히려 비중이 커졌다                                                             |
| 흐름도 배치 계산     | 그대로 쓴다 | 판정을 없을 캔버스가 필요하다                                                                   |
| 인수인계 문서 만들기   | 쓰지만 뒤로 | 1단계 이유였다가 4단계 부산물이 됐다                                                              |
| 자동화 후보 점수 매기기 | 대체된다   | 점수가 아니라 판정이다. 순위를 매기는 게 아니라 단계마다 넘길 수 있는지를 답한다. 반복 횟수·소요시간 계산은 재활용하지만 결과물의 형태가 다르다 |
| 판정            | 아직 없다  | 이번 논의로 방향이 정해졌고, 설계는 이 문서가 전부다                                                     |
| 화면            | 아직 없다  | 한 장도 없다                                                                            |

기획 문서는 26편이 있고, 계산 부분은 자동 점검 784가지를 통과한다. 다만 위 표대로 그중 하나는 방향 전환으로 형태가 바뀐다. 만들어 둔 게 헛것이 되지는 않았지만, 다 쓰이는 것도 아니다.

## 만들어 보니 기획이 틀렸던 것들

계산 부분을 실제로 만들면서, 문서만 쓸 때는 절대 안 나오는 오류가 드러났다. 지금까지 24건이고 전부 재발 방지 장치를 걸어 뒀다. 성격이 다른 셋만 옮긴다.

|                 |                                                                                                                                                |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 두 사람이 갈라진다      | 두 사람이 같은 문서를 동시에 고칠 때, 같은 위치에 각자 한 줄씩 넣으면 화면 A와 화면 B에 서로 다른 문장이 남았다. 충돌 표시가 안 뜬 게 아니라 결과 자체가 갈라진 것이다                                           |
| 원칙에 반례가 있었다     | "고친 단계를 화면에 붙잡아 둔다"는 원칙이 항상 옳지는 않았다. 움직인 게 그 단계 하나뿐일 때는, 그걸 붙잡으려고 화면을 미는 바람에 나머지가 전부 움직였다. 안 하느니만 못했다                                          |
| 적어 둔 식과 결과가 달랐다 | 예시 데이터에서 공식대로 계산하면 378만 원인데 카드에는 270만 원이 찍혀 있었다. 문서만 볼 때는 둘 다 그럴듯해서 아무도 안 잡았다. 판정에도 같은 일이 생길 수 있다. 근거는 맞는데 결론이 다른 경우다. 그래서 3장에서 근거를 항상 펼쳐 보여준다 |

## 아직 안 정한 것

|              |                                                                                                                |
|--------------|----------------------------------------------------------------------------------------------------------------|
| 판정을 무엇으로 돌리나 | 근거 세 가지는 정했다(2장). 그런데 이걸 실제로 어떻게 계산할지는 아직이다. 사내 도구 연결 표와 결재 규칙을 어떤 형태로 들고 있을지가 핵심인데, 여기가 이 프로젝트에서 가장 안 풀린 부분이다 |
| 실행을 어떻게 붙이나  | "지금 해볼까요"를 눌렀을 때 실제로 무엇이 도는지. 에이전트 연결 방식은 후보가 있지만 확정은 1단계 붙여 보고 정한다                                            |
| 공개 시점        | 사내 검증 후로 정했지만 "검증됐다"의 기준이 아직 없다. 5장 지표 중 무엇을 넘겨야 공개할지 정해야 한다                                                   |

## 다음에 할 일

가장 얇은 한 줄기를 처음부터 끝까지 연결한다.

질문 세 개에 답하기 → 단계 세 개 → 판정 → 그중 하나를 실제로 넘기기.

지금 가장 확인이 안 된 위험은 계산이 맞느냐가 아니라 **판정이 쓸 만하냐**다. 이걸 코드를 더 읽어서는 안 풀리고, 실제 업무 하나에 돌려 봐야 한다. 그리고 위에 적은 그 한 줄기가 7장의 가설과 정확히 같은 경로다. **이 한 줄에 프로젝트의 위험이 전부 한 번씩 걸려 있다.**

## 이 문서가 만들어진 방식

사람 한 명과 Claude가 함께 썼다. 기획자·디자이너·데이터 담당 등 21개의 서로 다른 역할로 나눠 검토를 반복했고, 각 역할은 다른 역할의 결론을 모른 채 판단했다. 의견이 갈린 지점은 사람이 정했고, 그 근거가 결정 기록 79건으로 남아 있다. 외부 전문가가 개별적으로 검토한 것은 아니다.

이 방식의 한계도 적어 둔다. 역할을 아무리 나뉘도 결국 같은 모델의 같은 편향을 공유한다. 그래서 확인할 수 있는 것은 전부 실제로 만들어서 돌려 봤다. 8장의 "만들어 보니 기획이 틀렸던 것들"은 검토를 더 해서가 아니라 실행해 봐서 나온 결과다.

상세 문서 26편과 코드는 여기에 있다 · [github.com/wlsdks/workflow-builder](https://github.com/wlsdks/workflow-builder)